

Wprowadzenie

1. Wprowadzenie do Dockera

Czym jest Docker?

Docker to platforma do konteneryzacji, która umożliwia uruchamianie aplikacji w lekkich, izolowanych środowiskach zwanych kontenerami. Kontenery są szybsze i bardziej elastyczne niż tradycyjne maszyny wirtualne.

Dlaczego Docker?

- **Izolacja:** Każdy kontener działa niezależnie od innych.
- **Portabilność:** Kontenery można przenosić między systemami.
- **Skalowalność:** Łatwe wdrażanie aplikacji w środowiskach produkcyjnych.

Następna strona



AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA
W KRAKOWIE

Platforma obsługiwana przez:
Centrum e-Learningu i
Innowacyjnej Dydaktyki AGH
Centrum Rozwiązań
Informatycznych AGH

Pobierz aplikację
mobilną



Wybierz
język



Instalacja i konfiguracja

2. Instalacja i konfiguracja Dockera

Docker powinien działać bez problemów, jeśli system jest odpowiednio skonfigurowany. Poniżej znajduje się szczegółowa instrukcja instalacji i konfiguracji.

Krok 1: Aktualizacja systemu

Na początek zaktualizuj pakiety systemowe:

```
sudo apt update && sudo apt upgrade -y
```

Krok 2: Instalacja Dockera

1. Pobierz wymagane zależności:

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common -y
```

2. Dodaj klucz GPG repozytorium Dockera (można użyć debiana zamiast ubuntu):

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-key
```

3. Dodaj repozytorium Dockera:

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubun
```

4. Zainstaluj Dockera:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io -y
```

Krok 3: Sprawdzenie instalacji

Sprawdź, czy Docker działa poprawnie:

```
sudo systemctl status docker
```

Sprawdź wersję Dockera:

```
docker --version
```

Krok 4: Dodanie użytkownika do grupy Docker (opcjonalnie)

Aby uruchamiać Dockera bez użycia **sudo**:

```
sudo usermod -aG docker $USER
```

UWAGA: Zmiany wejdą w życie po wylogowaniu i ponownym zalogowaniu.



Platforma obsługiwana przez:
Centrum e-Learningu i
Innowacyjnej Dydaktyki AGH
Centrum Rozwiązań
Informatycznych AGH

Pobierz aplikację
mobilną



Wybierz
język



Uruchamianie kontenerów

3. Uruchamianie kontenerów i podstawowe operacje

Krok 1: Uruchomienie pierwszego kontenera

1. Pobierz obraz `hello-world`:

```
docker pull hello-world
```

2. Uruchom kontener:

```
docker run hello-world
```

3. Wynik powinien pokazać komunikat "Hello from Docker!".

Krok 2: Zarządzanie kontenerami

1. Uruchom kontener w tle (tryb `detached`):

```
docker run -d --name my-nginx -p 8080:80 nginx
```

2. Sprawdź listę uruchomionych kontenerów:

```
docker ps
```

3. Zatrzymaj kontener:

```
docker stop my-nginx
```

4. Usuń kontener:

```
docker rm my-nginx
```

Krok 3: Konfiguracja sieci bridge

Kontenery korzystają domyślnie z bridge'a. Sprawdź konfigurację:

```
docker network inspect bridge
```



**AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA
W KRAKOWIE**

Platforma obsługiwana przez:
Centrum e-Learningu i
Innowacyjnej Dydaktyki AGH
Centrum Rozwiązań
Informatycznych AGH

Pobierz aplikację
mobilną



Wybierz
język



Dockerfile

Dockerfile

Aby zbudować obraz niezbędny jest poprawny Dockerfile, który opisuje każdą z warstw abstrakcji oraz metadane, które znajdują się w wyjściowym obrazie.

Typowy Dockerfile dla aplikacji nodejs może wyglądać tak:

```
MAINTAINER Jan Kowalski
LABEL "ocena"="5 gwiazdek" "klasa"="pierwsza"
USER root
ENV AP /data/app
ENV SCPATH /etc/supervisor/conf.d
RUN apt -y update
RUN apt -y install supervisor
RUN mkdir -p /var/log/supervisor
ADD .supervisor/conf.d/* $SCPATH
ADD *.js* $AP/
WORKDIR $AP
RUN npm install
CMD ["supervisord", "-n"]
FROM node:0.10
```

Każda z linii nakłada na obraz kolejną warstwę abstrakcji, dobrą praktyką jest zaczynać o najniższego poziomu jako od systemu, gdyż w przypadku zmian, np ścieżki w zmiennej AP, budowa nowego obrazu rozpocznie się od zmienionej linii, wszystkie wcześniejsze zostaną pominięte, co zaoszczędzi czasu przy budowaniu.

Poniżej wyjaśnię anatomię pliku Dockerfile na powyższym przykładzie.

- **FROM node:0.10** – oznacza, że nasz obraz będzie budowany na podstawie obrazu node w wersji 0.10.X, by zablokować do konkretnej wersji trzeba ją uściślić, np.: node:0.10.33.
- **MAINTAINER Jan Kowalski** – pozwala dodać kontakt do autora, propaguje autora również w metadanych wszystkich tworzonych obrazów.
- **LABEL „ocena”=„5 gwiazdek” „klasa”=„pierwsza”** – ta funkcjonalność istnieje od dockera 1.6, pozwala dodawać metadane w formie klucz=wartość, można na ich podstawie potem wyszukiwać i identyfikować kontenery
- **USER root** – domyślnie docker uruchamia wszystkie procesy w kontenerze jako root, za pomocą USER można to zmienić (ze względów bezpieczeństwa nie polecam produkcyjnie uruchamiać kontenerów z procesami roota, przypominam, że pomimo izolacji, nadal korzystamy z jądra gospodarza)
- **ENV AP /data/app** oraz **ENV SCPATH /etc/supervisor/conf.d** – zmienne, które mogą być użyte podczas budowy obrazu
- **RUN apt -y update / RUN apt -y install supervisor / RUN mkdir -p /var/log/supervisor** – polecenie RUN uruchamia jakąś instrukcję podczas budowy obrazu, np żeby zainstalować określone oprogramowanie czy zapewnić zależności.
- **ADD .supervisor/conf.d/* \$SCPATH / ADD *.js* \$AP/** – ADD kopiuje pliki z lokalnego systemu do obrazu
- **WORKDIR \$AP / RUN npm install** – WORKDIR zmienia aktualny katalog w jakim pracuje podczas budowania obrazu dla kolejnych instrukcji
- **CMD [„supervisord”, „-n”]** – CMD definiuje polecenie uruchamiane razem kontenerem (właściwą aplikację)

Każda kolejna linia Dockerfile i każdy krok tworzy nową warstwę, która zależy od poprzedniej!

Budowanie obrazu

Obrazy dockera budujemy za pomocą polecenia uruchamianego w katalogu z Dockerfile:

```
docker build -t example/docker-node-hello:latest .
```

Dockerfile proszę zakładać w katalogu domowym użytkownika lub jego podkatalogach.

Następna strona



Platforma obsługiwana przez:
Centrum e-Learningu i
Innowacyjnej Dydaktyki AGH
Centrum Rozwiązań
Informatycznych AGH

Pobierz aplikację
mobilną



Wybierz
język



Dockerfile cz 2

Struktura i Podstawowe Instrukcje Dockerfile

1. Instrukcja FROM

Określa podstawowy obraz, na którym zostanie oparty nowy obraz Dockera.

```
FROM ubuntu:20.04
```

W tym przykładzie używany jest obraz `ubuntu` w wersji `20.04` jako baza.

2. Instrukcja RUN

Pozwala wykonywać polecenia w trakcie budowy obrazu, takie jak instalacja oprogramowania lub modyfikacja systemu.

```
RUN apt update && apt install -y nginx
```

Instaluje serwer Nginx w obrazie.

3. Instrukcja COPY i ADD

Służą do kopiowania plików z hosta do obrazu Dockera:

- **COPY**: Kopiuje pliki w sposób prosty.
- **ADD**: Dodatkowo obsługuje rozpakowywanie archiwów.

```
COPY index.html /usr/share/nginx/html/index.html
```

Kopiuje plik `index.html` do katalogu serwera Nginx w kontenerze.

4. Instrukcja WORKDIR

Ustawia katalog roboczy w kontenerze, w którym będą wykonywane kolejne instrukcje.

```
WORKDIR /app
```

Ustawia `/app` jako domyślny katalog roboczy.

5. Instrukcja CMD

Definiuje domyślne polecenie, które zostanie wykonane po uruchomieniu kontenera. Może być nadpisane podczas uruchamiania kontenera.

```
CMD ["nginx", "-g", "daemon off;"]
```

Uruchamia serwer Nginx w trybie pierwszoplanowym.

6. Instrukcja ENTRYPOINT

Podobnie jak **CMD**, ale bardziej ogranicza możliwość nadpisania polecenia podczas uruchamiania kontenera.

```
ENTRYPOINT ["python3", "app.py"]
```

Ustawia `python3 app.py` jako niezmiennie polecenie startowe.

7. Instrukcja **ENV**

Ustawia zmienne środowiskowe wewnątrz kontenera.

```
ENV APP_ENV=production
```

Tworzy zmienną **APP_ENV** o wartości **production**.

8. Instrukcja **EXPOSE**

Deklaruje porty, które będą dostępne w kontenerze.

```
EXPOSE 80
```

Deklaruje, że kontener używa portu 80.

9. Instrukcja **VOLUME**

Definiuje punkt montowania, aby umożliwić przechowywanie danych poza kontenerem.

```
VOLUME /data
```

Tworzy punkt montowania **/data**.

10. Instrukcja **ARG**

Pozwala przekazywać zmienne podczas budowania obrazu.

```
ARG BUILD_VERSION=1.0
```

```
RUN echo "Build version: $BUILD_VERSION"
```

Przykład: Tworzenie prostego Dockerfile

Poniżej znajduje się przykład **Dockerfile** dla aplikacji opartej na Nginx, która serwuje plik **index.html**.

Dockerfile

```
# Użyj podstawowego obrazu
FROM nginx:latest

# Ustaw zmienną środowiskową
ENV APP_ENV=production

# Skopiuj plik HTML do obrazu
COPY index.html /usr/share/nginx/html/index.html

# Ustaw port
EXPOSE 80

# Domyślne polecenie
CMD ["nginx", "-g", "daemon off;"]
```

Kroki budowy obrazu i uruchomienia kontenera

1. Zbuduj obraz Dockera:

```
docker build -t my-nginx .
```

2. Uruchom kontener z obrazu:

```
docker run -d -p 8080:80 my-nginx
```

3. Otwórz w przeglądarce: <http://localhost:8080>

Zaawansowane Funkcje Dockerfile

1. Optymalizacja obrazu

Unikaj tworzenia dużych obrazów Dockera, stosując jeden **RUN** z wieloma poleceniami:

```
RUN apt update && apt install -y \  
  
    curl \  
    git \  
    && apt clean
```

2. Wieloetapowe budowanie

Stosuj wiele etapów, aby oddzielić środowisko budowania od środowiska produkcyjnego:

```
# Etap 1: Budowanie aplikacji  
FROM node:14 as builder  
WORKDIR /app  
COPY package.json .  
RUN npm install  
COPY . .  
RUN npm run build  
  
# Etap 2: Serwowanie aplikacji  
FROM nginx:alpine  
COPY --from=builder /app/build /usr/share/nginx/html  
CMD ["nginx", "-g", "daemon off;"]
```

Podsumowanie

Dockerfile to potężne narzędzie pozwalające na automatyzację procesu budowy obrazów Dockera. Znajomość jego składni i możliwości jest kluczowa dla każdego, kto chce efektywnie pracować z konteneryzacją. Dzięki Dockerfile można szybko budować, wdrażać i zarządzać aplikacjami w złożonych środowiskach.

Koniec

